

Rashomon — Architecture

Aseem Ratha

September 25, 2026

Contents

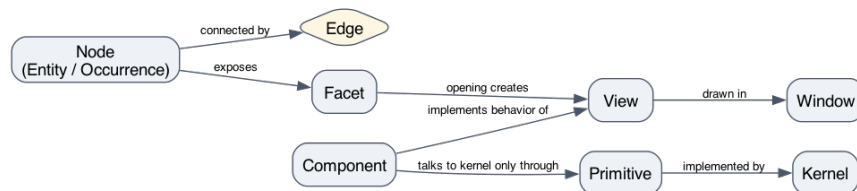
A desktop environment built around a persistent graph of information rather than conventional applications and files. Anything encountered while computing — webpages, notes, conversations, terminal sessions, media — can become a persistent, addressable thing in the graph. Programs become composable, swappable ways of encountering and manipulating that graph, not owners of their own private state.

Objects persist independently of the programs that operate on them; programs become composable ways of encountering and manipulating those objects.

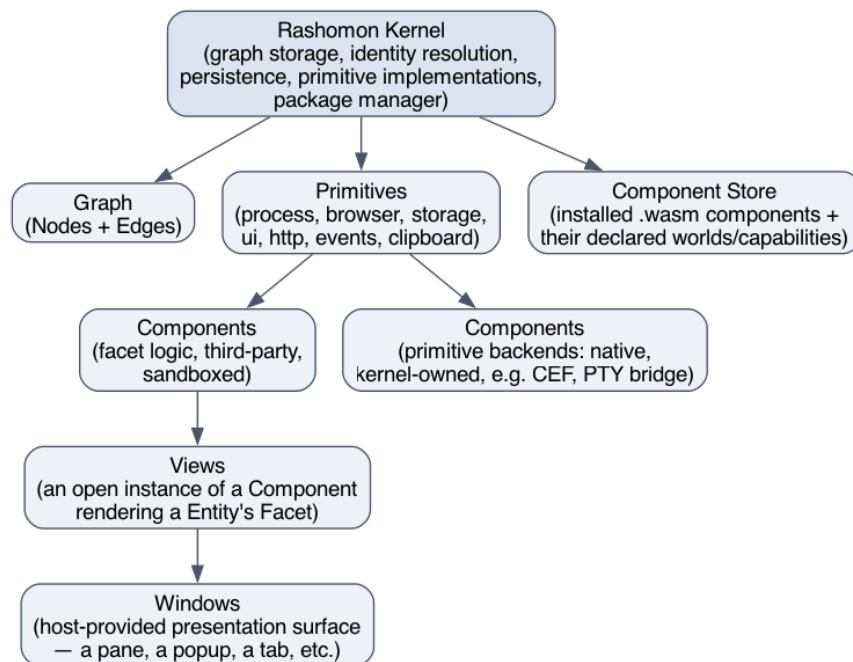
1. Core model

Term	Meaning
Rashomon	the whole system
Kernel	the native process owning the graph, primitives, persistence, and the component runtime
Node	the generic graph vertex. Every Node has a role : entity or occurrence
Entity	a Node with durable identity — a recurring page, a task, a topic, a browser profile
Occurrence	a Node representing one instance of running into something — this tab load, this shell session, this chat exchange. Weak/no identity of its own; always has exactly one occurrence-of edge to a Entity
Edge	a typed, directed link between two Nodes, carrying a timestamp and a confidence score
Facet	a declared way a Entity can be viewed or acted on (browser, notes, history, graph, terminal...)
View	one live, on-screen instance of a Facet being used right now. Facet = capability; View = an open instance of using it
Component	the WASM binary implementing a Facet's logic, or a primitive's native backend
Window	the OS-level surface a View is drawn in
Primitive	a host-implemented capability a Component can be granted access to (graph read/write, process spawn, browser control, network, storage...)

One sentence: **the graph is made of Nodes — Entities and Occurrences — connected by Edges; a Entity exposes Facets; opening a Facet creates a View; a View is drawn in a Window; a View's behavior is implemented by a Component; Components talk to the kernel only through Primitives.**



2. Layered overview



Hosts (an extension, an Electron/CEF shell, eventually a browser fork) sit outside this entirely — they only provide Windows and route user input in. Swapping a host should never touch the graph, the primitives, or any Component.

3. Nodes: roles vs. types

entity and **occurrence** are the only two **roles**, fixed by the kernel — this is what determines identity-resolution behavior and which structural edges apply. Concrete node **types** are open and component-defined, the same way Facets are.

A component's manifest registers the node types it introduces:

```
node-type shell-session {  
  role: occurrence  
  properties: { cwd: string, started-at: timestamp, exit-code: option<s32> }  
}
```

Types are namespaced (`rashomon:page`, `someauthor:shell-session`) so two components can't collide on the same name.

Built-in types (ship with the kernel)

Type	Role	Notes
<code>thread</code>	entity	generic, no-schema — the default place to file anything with no more specific type
<code>page</code>	entity	a recurring web reference; identity resolved via URL + content similarity
<code>browser-context</code>	entity	a persistent browser profile/session identity

Example third-party types

Type	Role	Notes
<code>shell-session</code>	occurrence	one PTY-backed terminal run
<code>dev-task</code>	entity	optional richer entity a terminal component could define if generic <code>thread</code> isn't enough (tracks cwd/branch across sessions)
<code>chat-thread</code>	entity	a line of inquiry with an LLM
<code>chat-exchange</code>	occurrence	one specific conversation instance under a <code>chat-thread</code>
<code>installed-component</code>	entity	the kernel's own package manager represents installed Components as ordinary Nodes — self-hosting in practice

4. Edges

Structural (kernel reasons about these directly):

- `occurrence-of` — Occurrence $\rightarrow$ Entity. Mandatory, exactly one per Occurrence.
- `contains` — Entity $\rightarrow$ Entity. The outline/tree hierarchy (threads containing pages, pages containing markers). Single-parent by default — a strict tree, not a DAG, so outline-style Facets can render it simply. A second relationship uses `references` instead.

- **supersedes / diverged-from** — Entity → Entity. Produced by the identity-resolution ladder when a fuzzy match falls below the merge threshold, so a bad automatic call is correctable, not silently wrong.

Typed, open (kernel stores/queries generically; Components interpret):

- **references** — general "this points at that." An anchor is a specialization: **references** carrying an **anchor-spec** payload (quote + context, element descriptors, spatial position, or a media locator).
- **context-of** — Occurrence → Entity, for "this happened under this browser-context/profile."
- **derived-from** — provenance for generated content (a chat reply derived from the anchors in its context; a promoted note derived from a message).
- **related-to** — a deliberately weak, manual "these matter to each other."

Every edge carries a **timestamp** and a **confidence score** (1.0 for anything manual; a real number from the similarity check for automatic **supersedes** edges).

5. Identity: entities and occurrences

Resolution is a per-occurrence matching problem, not a global definition of "what a webpage is." A **ladder**, run per occurrence against candidates a Facet's own matcher proposes:

1. **Strong key match** — a stable identifier (content hash, commit SHA, message ID). Exact match → same Entity, new version if content differs.
2. **Normalized-key match + similarity check** — e.g. a normalized URL. If a Entity already claims the key, diff new content against the last version. Above threshold → new version of the same Entity. Below threshold → new Entity, linked via **supersedes**~/~/**diverged-from**.
3. **No match** → new Entity.

Keys are typed and Facet-defined, not hardcoded in the kernel — a browser Facet proposes URL-based keys, a terminal Facet proposes directory+branch keys, a chat Facet proposes participant+topic keys. Every merge/split is logged and reversible.

6. Primitives

A fixed, versioned WIT package every Component’s **world** is composed from. Semver’d independently of the kernel’s internals — this is the contract a future maintainer only needs to keep stable, not every Facet ever published.

- **rashomon:graph** — read/write Nodes and Edges; propose-match calls for the identity ladder.
- **rashomon:storage** — a sandboxed scope per Component.
- **rashomon:process** — spawn/manage OS processes via a **resource process** handle (stream in/out, resize, signal, wait). The kernel owns the actual `exec()`; a Component never gets a raw syscall.
- **rashomon:browser** — **resource browser-context** (a persistent profile: cookies, cache, logins — itself represented as a **browser-context** Entity) and **resource browser-view** (navigate, DOM snapshot, inject script, anchor, downloads). Backed natively by CEF; never embedded in WASM itself (too heavy, needs native GPU/windowing).
- **wasi:http** — standard, reused rather than wrapped.
- **rashomon:ui** — a render-surface contract: a Component hands back a description of what to draw; the host draws it.
- **rashomon:signals** — subscribe to graph changes, timers, lifecycle.
- **rashomon:clipboard** / **rashomon:notify** — small, obviously-scoped.

A Component’s declared world (its imports) is static and inspectable before install — the permission-prompt story, shown the way an extension’s requested permissions are shown today.

Decide `rashomon:process` and `rashomon:browser` against a real Facet that needs them, not in the abstract — designing a capability before the first consumer exists is a common way these interfaces end up wrong.

7. Self-hosting

State lives in the **graph**, not inside a Component instance — a Component is closer to a stateless request handler over persistent Nodes than a live object with private state. That's what makes **hot-swap** (replace a running instance with a freshly compiled one) practical, even though true Lisp/Smalltalk-style **hot-edit** (mutating running code in place) isn't available to a compiled, sandboxed WASM binary.

The core design rule: **anything the kernel needs to show or manage about itself must first be representable as Nodes and Edges**, so managing it can be delegated to an ordinary Component. The node-type registry browser, the package manager, and the outline/thread view should all be installed Components from day one — not privileged native UI — or the system never actually becomes self-hosting later.

A REPL/scripting Facet is a plausible later addition (Nyxt's Lisp REPL, Emacs's **eval-buffer** are the precedent) but is **deferred** — not required while Components can already be written in any supported language, and worth designing only once the friction of not having one is concrete.

8. Packaging and distribution

A package = one **.wasm** file + a manifest declaring node types, Facets, edge types, and required primitives — the direct equivalent of a **.vsix**.

- **Local dev loop:** load straight from a directory (VS Code's "install from location" pattern) — no registry round-trip while iterating.
- **Distribution:** skip **warg**~/~**wa.dev** for now (still early) in favor of a signed-manifest index hosted on GitHub, browsed through an Extensions Facet (itself a Component) that shows requested capabilities before install — fetch, verify hash/signature, unpack, register. No config file editing, ever, for the install path itself.
- Revisit **warg** once it matures; the manifest format should be designed so migrating onto it later doesn't require re-authoring packages.

9. Concurrency

- **Across Views/Components:** each active View gets its own Wasm-time **Store**. I/O-bound work (watchers, streaming chat, browser events)

runs as async tasks sharing an executor (Wasmtime's component-model-async support). CPU-bound work dispatches its `Store`'s call onto a worker thread so it doesn't stall the executor.

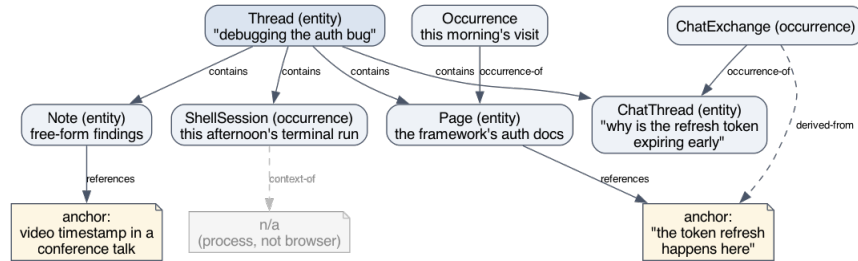
- **Within a single Component:** true in-guest multithreading needs the WebAssembly "shared-everything-threads" proposal, which is still immature. **Don't build around it.** Instead, a Component that needs real parallel compute calls a **primitive** implemented natively (multithreaded Rust on the host side) and awaits the result — the same host-mediated pattern as `rashomon:process` and `rashomon:browser`.

10. Tech stack

- **Kernel:** Rust.
- **WASM runtime:** Wasmtime — the Bytecode Alliance reference implementation, furthest along on the Component Model and WASI Preview 2.
- **Browser primitive backend:** CEF via `cef-rs` (the Tauri-maintained fork) — actively tracked against current Chromium releases, ships its own binary-download and app-bundling tooling.
- **Component authoring:** any language with WIT bindings. Rust (`cargo-component`) is most mature; JS (`jco`), Python (`componentize-py`), and Go (TinyGo) are workable; C/C++ guest support is real but currently the least polished.
- **UI:** HTML-based, described by Components through `rashomon:ui` and drawn by the host — not native per-toolkit UI code.

11. Worked example

A thread called **"debugging the auth bug"**:



Reopening the Thread surfaces every page, session, and chat filed under it, in the order and structure you left them — "resume" is a graph query, not a manually rebuilt context.

12. Open / deferred decisions

- Whether `rashomon:process` ships in v1 or waits for the first real terminal Facet to design against.
- Same question for `rashomon:browser`'s multi-context isolation — build single-context/single-view first.
- Scripting/REPL Facet — deferred until the friction is concrete.
- `warg` adoption — revisit once it's past its early stage.
- Whether `contains` should ever allow multiple parents — currently single-parent by design, with `references` covering secondary links.